System Design Document

Placement Automation Tool

1. Introduction

1.1 Purpose

This document describes the **system architecture, design components, database structure, APIs, and workflows** of the **Placement Automation Tool (PAT)**. The system is designed to automate the recruitment drive management process for universities by allowing administrators, recruiters, and students to manage placement drives efficiently.

The system supports **user authentication, recruitment drive management, eligibility filtering, student applications, and analytics dashboards**.

1.2 Scope

The system enables the following:

- Secure **user authentication using JWT**
- **Creation and management of recruitment drives**
- **Student eligibility filtering**
- **Student application workflow with stage tracking**
- **Conversion ratio metrics**
- **React dashboard for viewing drives and applications**
- **Role-based access control**

1.3 Stakeholders

The system serves multiple stakeholders:

Role	Description
Student	Applies for recruitment drives
Coordinator	Can validate and remove students, post announcements too
TPO	Can schedule and add drives
Admin	Manages users and system

---

2. System Overview

The **Placement Automation Tool** is a **web-based recruitment management system** consisting of:

- **React Frontend**
- **Spring Boot Backend**
- **MySQL Database**

The system allows companies to post recruitment drives, students to apply, and administrators to monitor recruitment metrics.

---

3. Technology Stack

Backend

Layer	Technology
Language	Java 21
Framework	Spring Boot 3.3.5
Security	Spring Security + JWT
ORM	Spring Data JPA + Hibernate
Database	MySQL 8
Build Tool	Maven
API Testing	Postman
IDE	Eclipse

Frontend

Layer	Technology
Framework	React 19 (Vite)
HTTP Client	Axios
Routing	React Router DOM v7
Styling	CSS

4. System Architecture

The system follows a **Layered Architecture with REST APIs**.

```
Browser (React UI - Port 5173)
        |
        ▼
HTTP Requests (Axios via Vite Proxy /api → 8080)
        |
        ▼
Spring Boot Backend (Port 8080)

Controller Layer
(AuthController, UserController, DriveController, etc.)
        |
        ▼
Service Layer (Business Logic)
(AuthService, DriveService, ApplicationService, StudentServic
e)
        |
        ▼
Security Layer
(JWT Filter, JwtUtil, SecurityConfig)
        |
```

```
        ▼
DTO Layer
(Request/Response Objects for API communication)
        │
        ▼
Repository Layer
(JPA Repositories for DB access)
        │
        ▼
Entity Layer (Model Layer)
(User, Student, Drive, Application, etc.)
        │
        ▼
MySQL Database
```

High-Level Architecture

```
Browser (React - port 5173)
        │
        ▼ HTTP via Vite Proxy (/api → 8080)
┌──────────────────────────────────────────────────┐
│              Controller Layer                    │
│  AuthController                                  │
│  UserController                                  │
│  ContentController                               │
│  DriveController                                 │
│  TpoDriveController                              │
│  ApplicationController                           │
│  StudentController                               │
│  StudentProfileController                        │
└──────────────────┬───────────────────────────────┘
                   │
                   ▼
┌──────────────────────────────────────────────────┐
```

```
│                Service Layer                  │
│  AuthService                                  │ ← OTP + Emai
l Verification
│  EmailService                                 │ ← Gmail SMTP
Integration
│  DriveService                                 │
│  ApplicationService                           │ ← Eligibilit
y Engine + Metrics
│  StudentService                               │
└──────────────────────────────────────────────┘
                 │
                 │
                 ▼
┌──────────────────────────────────────────────┐
│              Security Layer                   │
│  JwtAuthenticationFilter                      │
│  JwtUtil                                      │
│  SecurityConfig                               │
└──────────────────────────────────────────────┘
                 │
                 │
                 ▼
┌──────────────────────────────────────────────┐
│                DTO Layer                      │
│  SignupRequest, LoginRequest                  │
│  AuthResponse, ApiResponse                    │
│  ForgotPasswordRequest, ResetPasswordRequest  │
│  ConversionMetrics                            │
└──────────────────────────────────────────────┘
                 │
                 │
                 ▼
┌──────────────────────────────────────────────┐
│             Repository Layer                  │
│  UserRepository                               │
│  DriveRepository                              │
│  StudentRepository                            │
│  StudentProfileRepository                     │
│  ApplicationRepository                        │
```

```
│  ContentItemRepository                          │
│  DriveStudentRepository                         │
│  PatDriveRepository                             │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
┌─────────────────────────────────────────────────┐
│                Entity Layer                     │
│  User (otp, otpExpiry)                          │
│  Role (STUDENT, COORDINATOR, TPO, ADMIN, HR)│
│  Drive (eligibility fields)                     │
│  Student (graduationYear, skills)               │
│  StudentProfile                                 │
│  Application                                    │
│  ContentItem                                    │
│  DriveStudent                                   │
│  PatDrive                                       │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
┌─────────────────────────────────────────────────┐
│             Exception Handling                  │
│  GlobalExceptionHandler                         │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
┌─────────────────────────────────────────────────┐
│              Database (MySQL)                   │
│  users                                          │
│  roles                                          │
│  drives                                         │
│  students                                       │
│  student_profiles                               │
│  applications                                   │
│  content_items                                  │
│  drive_students                                 │
```

```
│  pat_drives                                          │
└──────────────────────────────────────────────────────┘
```

5. Team Responsibilities

Module	Responsibilities
Authentication Module	Signup, Login, JWT, Password Reset
Recruitment Module	Drive CRUD, Eligibility Engine, Applications
Analytics Module	Shows reports like-Average package,etc

6. Security Architecture

The system uses **JWT-based authentication**.

Authentication Flow

1. User registers or logs in
2. Backend generates **JWT token**
3. Token returned to frontend
4. Frontend stores token
5. Token sent in **Authorization Header**

Role-Based Access Control

Roles:

- STUDENT
- COORDINATOR
- TPO
- Admin

7. Database Design

Database Name:

```
recruitment_system
```

```
+----------------+
|     USERS      |
+----------------+
| id (PK)        |
| name           |
| email          |
| password       |
| role           |
+----------------+
        |
        | 1
        |
        |------< student_profile
        |
        |------< content_items
        |
        |------< drives
                      |
                      | 1
                      |
                      |------< drive_students


+----------------------+
|   STUDENT_PROFILE    |
+----------------------+
| id (PK)              |
| user_id (FK)         |
| phone                |
```

```
| linkedin             |
| address              |
| cgpa                 |
| department           |
| tech_skills          |
| soft_skills          |
+----------------------+

+----------------------+
|    CONTENT_ITEMS     |
+----------------------+
| id (PK)              |
| type                 |
| title                |
| body                 |
| coordinator_user_id  |
+----------------------+

+----------------------+
|       DRIVES         |
+----------------------+
| id (PK)              |
| tpo_user_id (FK)     |
| company              |
| role                 |
| drive_date           |
| venue                |
| eligibility          |
| rounds               |
| status               |
+----------------------+

+----------------------+
|    DRIVE_STUDENTS    |
+----------------------+
| id (PK)              |
```

```
| drive_id (FK)        |
| student_name         |
| student_email        |
| round_index          |
| status               |
+----------------------+
```

8. Eligibility Engine

The **Eligibility Engine** filters students based on drive requirements.

Eligibility criteria include:

- Minimum CGPA/Percentage
- Eligible branches
- Graduation year

TPO Drive Creation Process

```
TPO Logs In
     |
     ▼
Navigate to "Create Drive"
     |
     ▼
Enter Basic Information
(Company, Role, Job Type, CTC, Location)
     |
     ▼
Enter Drive Details
(Date, Time, Mode, Venue/Link)
     |
     ▼
Set Eligibility Criteria
(CGPA, 10th %, 12th %, Graduation Year, Branches)
```

```
     |
     ▼
Define Selection Process
(Aptitude, Coding, Technical, HR, GD)
     |
     ▼
Add Deadlines & Links
(Registration Deadline, Result Date, JD Link, Bond Details)
     |
     ▼
TPO Creates Drive
     |
     ▼
Drive Stored in Database with Criteria
     |
     ▼
Students View Drives
     |
     ▼
System Filters Drives Per Student
(Only eligible drives are shown)
     |
     ▼
Student Applies to Drive if Interested
     |
     ▼
Applied Student List Seen by TPO who Manages the Drive Funnel
```

---

9. Application Workflow

Student application process:

```
Student Login
      |
```

```
      ▼
View ONLY the Drives They are Eligible for
      |
      ▼
Apply for Drive
      |
      ▼
Application Created
      |
      ▼
Stage Tracking
(Aptitude → Technical → HR → Selected)
```

---

10. Conversion Ratio Metrics

The system calculates **recruitment analytics**.

Metrics include:

- Total applicants
- Selected candidates
- Round-wise pass percentage

Example:

```
Conversion Ratio = Selected Students / Total Applicants
```

This helps administrators analyze recruitment success.

11. Exception Handling

The system uses **GlobalExceptionHandler**.

Handles:

- Authentication errors

- Validation errors
- Database errors

---

12. Deployment Architecture

Deployment components:

```
Client Browser
      |
React Frontend (Vite)
      |
Spring Boot Backend
      |
MySQL Database
```

Ports used:

Component	Port
React	5173
Spring Boot	8080
MySQL	3306

---

13. Future Enhancements

Potential improvements include:

- Email notifications for recruitment stages
- Resume upload system
- Advanced analytics dashboard

---

14. Conclusion

The **Placement Automation Tool** provides an efficient platform for managing recruitment drives in educational institutions. By integrating **Spring Boot backend services, React frontend dashboards, and a MySQL database**, the system ensures secure authentication, automated eligibility filtering, streamlined application workflows, and recruitment analytics.